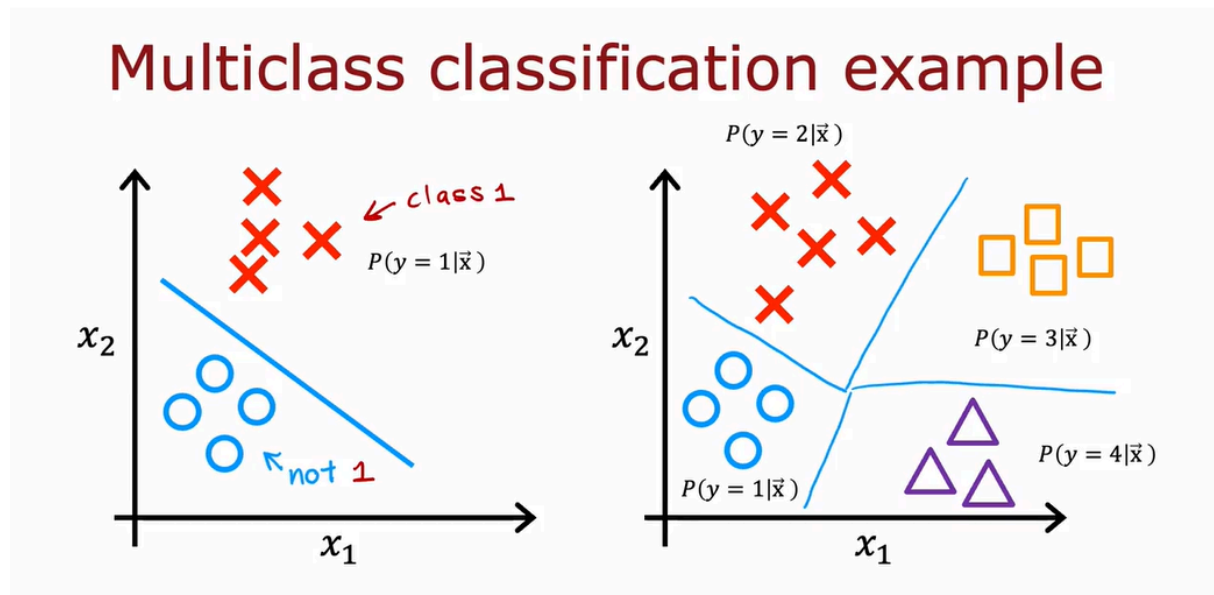


Multiclass Classification

Multiclass



Softmax regression

Logistic regression (2 possible output values) $z = \vec{w} \cdot \vec{x} + b$ ✗ $a_1 = g(z) = \frac{1}{1+e^{-z}} = P(y=1 \vec{x})$ 0.11 ○ $a_2 = 1 - a_1 = P(y=0 \vec{x})$ 0.29 Softmax regression (N possible outputs) $y=1,2,3,\dots,N$ $z_j = \vec{w}_j \cdot \vec{x} + b_j \quad j = 1, \dots, N$ parameters $w_1, w_2, \dots, w_N$ $b_1, b_2, \dots, b_N$ $a_j = \frac{e^{z_j}}{\sum_{k=1}^N e^{z_k}} = P(y=j \vec{x})$ note: $a_1 + a_2 + \dots + a_N = 1$	Softmax regression (4 possible outputs) $y=1,2,3,4$ ✗ $z_1 = \vec{w}_1 \cdot \vec{x} + b_1$ $a_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}}$ ✗ ○ □ △ $= P(y=1 \vec{x})$ 0.30 ○ $z_2 = \vec{w}_2 \cdot \vec{x} + b_2$ $a_2 = \frac{e^{z_2}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}}$ $= P(y=2 \vec{x})$ 0.20 □ $z_3 = \vec{w}_3 \cdot \vec{x} + b_3$ $a_3 = \frac{e^{z_3}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}}$ $= P(y=3 \vec{x})$ 0.15 △ $z_4 = \vec{w}_4 \cdot \vec{x} + b_4$ $a_4 = \frac{e^{z_4}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}}$ $= P(y=4 \vec{x})$ 0.35
--	--

Having defined how softmax regression computes its outputs, let's now take a look at how to specify the cost function for softmax regression.

Cost

Logistic regression

$$z = \vec{w} \cdot \vec{x} + b$$

$$a_1 = g(z) = \frac{1}{1 + e^{-z}} = P(y = 1 | \vec{x})$$

$$a_2 = 1 - a_1 = P(y = 0 | \vec{x})$$

$$\text{loss} = \underbrace{-y \log a_1}_{\text{if } y=1} - \underbrace{(1-y) \log(1-a_1)}_{\text{if } y=0}$$

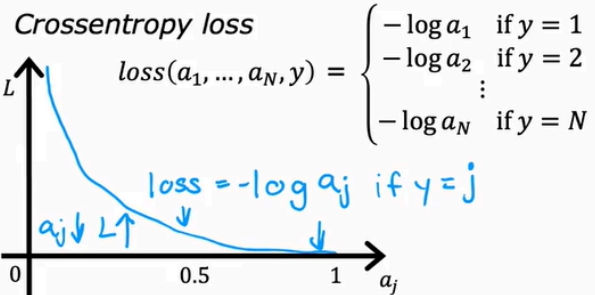
$$J(\vec{w}, b) = \text{average loss}$$

Softmax regression

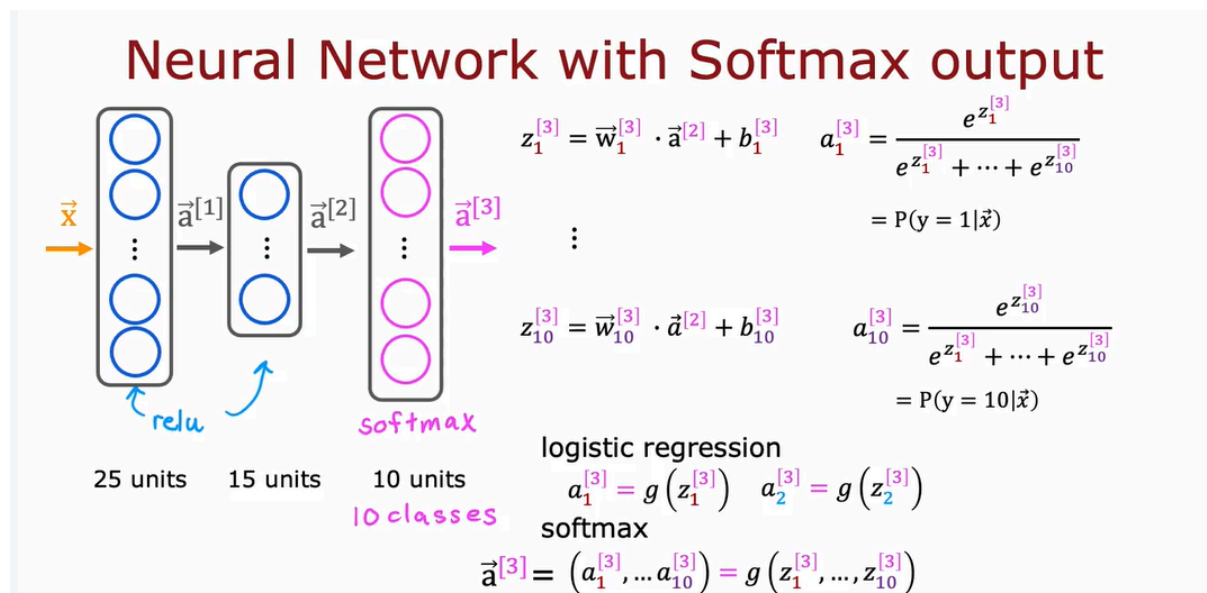
$$a_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2} + \dots + e^{z_N}} = P(y = 1 | \vec{x})$$

$$\vdots$$

$$a_N = \frac{e^{z_N}}{e^{z_1} + e^{z_2} + \dots + e^{z_N}} = P(y = N | \vec{x})$$



Neural Network with Softmax output



I do want to mention that the Softmax layer will sometimes also called the Softmax activation function, it is a little bit unusual in one respect compared to the other activation functions we've seen so far, like sigmoid, ReLU and linear, which is that when we're looking at sigmoid or value or linear activation functions.

let's look at how you would implement this in tensorflow.

MNIST with softmax

①

specify the model

$$f_{\vec{w},b}(\vec{x}) = ?$$

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
model = Sequential([
    Dense(units=25, activation='relu'),
    Dense(units=15, activation='relu'),
    Dense(units=10, activation='softmax')
])
```

②

specify loss and cost

$$L(f_{\vec{w},b}(\vec{x}), y)$$

```
from tensorflow.keras.losses import
    SparseCategoricalCrossentropy
model.compile(loss= SparseCategoricalCrossentropy() )
```

③

Train on data to
minimize $J(\vec{w}, b)$

```
model.fit(X, Y, epochs=100)
Note: better (recommended) version later.
```

So even though the code shown in this slide works. Don't use this code the way I've written it here, because in a later video this week you see a different version that's actually the recommended version of implementing this, that will work better. But we'll take a look at that in a later video.

Improved implementation of softmax

Numerical Roundoff Errors

option 1

$$x = \frac{2}{10,000}$$

option 2

$$x = \left(1 + \frac{1}{10,000}\right) - \left(1 - \frac{1}{10,000}\right)$$

```

In [1]: x1 = 2.0 / 10000
print(f"{x1:.18f}") # print 18 digits to the right of decimal point
0.00020000000000000000

In [2]: x2 = 1 + (1/10000) - (1 - 1/10000)
print(f"{x2:.18f}")
0.00019999999999999998

In [ ]:

```

It just looks a little bit off this as if there's some round-off error. Because the computer has only a finite amount of memory to store each number, called a floating-point number in this case, depending on how you decide to compute the value $2/10,000$, the result can have more or less numerical round-off error. It turns out that while the way we have been computing the cost function for softmax is correct, there's a different way of formulating it that reduces these numerical round-off errors, leading to more accurate computations within TensorFlow.

Numerical Roundoff Errors

More numerically accurate implementation of logistic loss: $1 + \frac{1}{10,000}$ $1 - \frac{1}{10,000}$

Logistic regression: $a = g(z) = \frac{1}{1 + e^{-z}}$

Original loss

$$\text{loss} = -y \log(a) - (1 - y) \log(1 - a)$$

More accurate loss (in code)

$$\text{loss} = -y \log\left(\frac{1}{1 + e^{-z}}\right) - (1 - y) \log\left(1 - \frac{1}{1 + e^{-z}}\right)$$

logit: z

```

model = Sequential([
    Dense(units=25, activation='relu'),
    Dense(units=15, activation='relu'), 'linear'
    Dense(units=1, activation='sigmoid')
])
model.compile(loss=BinaryCrossEntropy())
model.compile(loss=BinaryCrossEntropy(from_logits=True))

```

Let me first explain this a little bit more detail using logistic regression. Then we will show how these ideas apply to improving our implementation of softmax.

First, let me illustrate these ideas using logistic regression. Then we'll move on to show how to improve your implementation of softmax as well. Recall that for logistic regression, if you want to compute the loss function for a given example, you would first compute this output activation a (the letter a in red circle), which is $g(z) = \frac{1}{1+e^{-z}}$. Then you will compute the loss using this expression over here (the original loss formulation).

In fact, this is what the codes would look like for a logistic output layer with this binary cross entropy loss. For logistic regression, this works okay, and usually the numerical round-off errors aren't that bad. But it turns out that if you allow TensorFlow to not have to compute a as an intermediate term. But instead, if you tell TensorFlow that the loss this expression down here. All I've done is I've taken a and expanded it into this expression down here. Then TensorFlow can rearrange terms in this expression and come up with a more numerically accurate way to compute this loss function. Whereas the original procedure was like insisting on computing a as an intermediate value, 1 plus $1/10,000$ and another intermediate value, 1 minus $1/10,000$, then manipulating these two to get $2/10,000$.

This partial implementation was insisting on explicitly computing a as an intermediate quantity. But instead, by specifying this expression at the bottom directly as the loss function, it gives TensorFlow more flexibility in terms of how to compute this and whether or not it wants to compute a explicitly. The code you can use to do this is shown here and what this does is it sets the output layer to just use a linear activation function and it puts both the activation function, $\frac{1}{1+e^{-z}}$, as well as this cross entropy loss into the specification of the loss function over here. That's what this from logits equals true argument causes TensorFlow to do. In case you're wondering what the logits are, it's basically this number z .

TensorFlow will compute z as an intermediate value, but it can rearrange terms to make this become computed more accurately. One downside of this code is it becomes a little bit less legible. But this causes TensorFlow to have a little bit less numerical roundoff error.

More numerically accurate implementation of softmax

Softmax regression

$$(a_1, \dots, a_{10}) = g(z_1, \dots, z_{10})$$

$$\text{Loss} = L(\vec{a}, y) = \begin{cases} -\log a_1 & \text{if } y = 1 \\ \vdots \\ -\log a_{10} & \text{if } y = 10 \end{cases}$$

```
model = Sequential([
    Dense(units=25, activation='relu'),
    Dense(units=15, activation='relu'),
    Dense(units=10, activation='softmax')
])
```

'linear'

More Accurate

$$L(\vec{a}, y) = \begin{cases} -\log \frac{e^{z_1}}{e^{z_1} + \dots + e^{z_{10}}} & \text{if } y = 1 \\ \vdots \\ -\log \frac{e^{z_{10}}}{e^{z_1} + \dots + e^{z_{10}}} & \text{if } y = 10 \end{cases}$$

```
model.compile(loss=SparseCategoricalCrossEntropy())
```

```
model.compile(loss=SparseCategoricalCrossEntropy(from_logits=True))
```

MNIST stands for **Modified National Institute of Standards and Technology** dataset.

It's a classic benchmark dataset used in machine learning and computer vision, especially for training and testing image classification models.

MNIST (more numerically accurate)

```
model    import tensorflow as tf
         from tensorflow.keras import Sequential
         from tensorflow.keras.layers import Dense
         model = Sequential([
             Dense(units=25, activation='relu'),
             Dense(units=15, activation='relu'),
             Dense(units=10, activation='linear') ])

loss     from tensorflow.keras.losses import
         SparseCategoricalCrossentropy

fit      model.compile(..., loss=SparseCategoricalCrossentropy(from_logits=True))

predict logits = model(X) ← not a1...a10
         f_x = tf.nn.softmax(logits)      is z1...z10
```

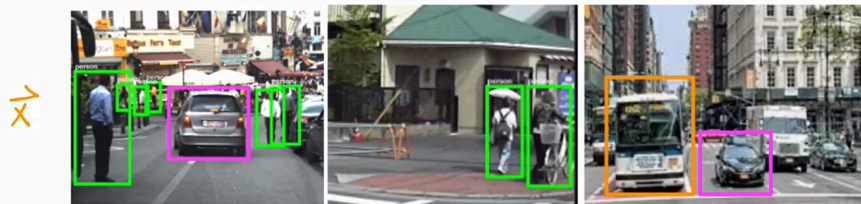
The neural network's final layer no longer outputs these probabilities a_1 through a_{10} . It is instead of putting z_1 through z_{10} .

logistic regression (more numerically accurate)

```
model    model = Sequential([
          Dense(units=25, activation='sigmoid'),
          Dense(units=15, activation='sigmoid'),
          Dense(units=1, activation='linear')
        ])
          from tensorflow.keras.losses import
          BinaryCrossentropy
loss      model.compile(..., BinaryCrossentropy(from_logits=True))
          model.fit(X,Y,epochs=100)
fit       logit = model(X)
predict   f_x = tf.nn.sigmoid(logit)
```

Classification with multiple outputs

Multi-label Classification



Is there a car?	yes	$y = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$	no	$y = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$	yes	$y = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}$
Is there a bus?	no		no		yes	
Is there a pedestrian	yes		yes		no	

You've learned about multi-class classification, where the output label Y can be any one of two or potentially many more than two possible categories. There's a different type of classification problem called a multi-label classification problem, which is where associate of each image, they could be multiple labels.

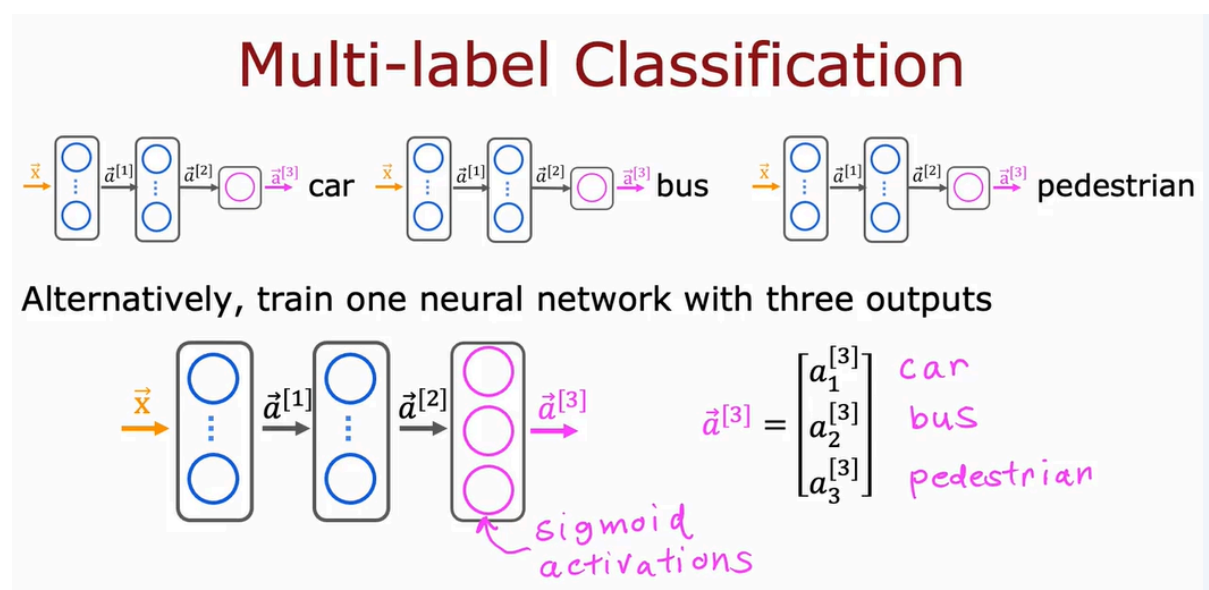
Let me show you what I mean by that. If you're building a self-driving car or maybe a driver assistance system, then given a picture of what's in front of your car, you may want to ask a question like, is there a car or at least one car? Or is there a bus, or is there a pedestrian or are there any pedestrians?

In this case (the most left case), there is a car, there is no bus, and there is at least one pedestrian or in this second image (the middle one), no cars, no buses and yes to pedestrians and (the most right one) yes car, yes bus and no pedestrians.

These are examples of multi-label classification problems because associated with a single input, image X are three different labels corresponding to whether or not there are any cars, buses, or pedestrians in the image.

In this case, the target of the Y is actually a vector of three numbers, and this is as distinct from multi-class classification, where for, say handwritten digit classification, Y was just a single number, even if that number could take on 10 different possible values.

How do you build a neural network for multi-label classification? One way to go about it is to just treat this as three completely separate machine learning problems. You could build one neural network to decide, are there any cars? The second one to detect buses and the third one to detect pedestrians.



First hidden layer offers $\vec{a}^{[1]}$, second hidden layer offers $\vec{a}^{[2]}$, and then the final output layer, in this case, we'll have three output neurals and we'll output $\vec{a}^{[3]}$, which is going to be a vector of three numbers. Because we're solving three binary classification problems, so is there a car? Is there a bus? Is there a pedestrian?

You can use a sigmoid activation function for each of these three nodes in the output layer, and so $\vec{a}^{[3]}$ in this case will be $\vec{a}_1^{[3]}$, $\vec{a}_2^{[3]}$, and $\vec{a}_3^{[3]}$, corresponding to

whether or not the learning algorithm think as a car and or bus, and or pedestrians in the image.

Multi-class classification and multi-label classification are sometimes confused with each other, and that's why in this video I want to share with you just a definition of multi-label classification problems as well, so that depending on your application, you could choose the right one for the job you want to do. So that's it for multi-label classification. I find that sometimes multi-class classification and multi-label classification are confused with other, which is why I wanted to expressively, in this video, share with you what is multi-label classification